

Tecnicatura Universitaria en Programación - Universidad Tecnológica Nacional.

Programación 1

Trabajo Práctico Integrador

Gestión de Datos de Países en Python: filtros, ordenamientos y estadísticas

Alumnos

Antonella Anahi Lopez - M26 C1-14

Grupo: 106

Junio de 2026

Tabla de contenido

Introducción 2

Objetivo 2

Marco Teórico3

 Listas

 Diccionarios

 Funciones

 Condicionales

 Ordenamientos

 Estadísticas Básicas

 Archivos CSV

Decisiones Técnicas y Arquitectura4

Desarrollo del Proyecto6

Capturas de Ejecución 7

Dificultades Encontradas9

Conclusiones 10

Bibliografía / Webgrafía11

Links del Proyecto12

Introducción

El presente trabajo práctico fue desarrollado con el objetivo de aplicar conceptos fundamentales de programación utilizando el lenguaje Python.

El sistema implementado permite gestionar información de distintos países mediante el uso de archivos de texto, listas, diccionarios y funciones. Además, el proyecto busca fortalecer conocimientos relacionados con la organización de datos, reutilización de código y persistencia de información.

El desarrollo fue realizado utilizando Visual Studio Code y posteriormente almacenado en GitHub para mantener un control de versiones adecuado.

Objetivo

General

Desarrollar un programa en Python capaz de leer, almacenar y procesar información de países utilizando estructuras de datos y archivos de texto.

Específicos

Comprender el funcionamiento de listas y diccionarios.

Aplicar funciones para modularizar el código.

Leer y escribir información en archivos de texto.

Organizar los datos de manera eficiente.

Implementar búsquedas y visualización de información.

Marco Teórico

Listas

Una lista es una estructura ordenada y mutable que almacena múltiples valores. En el proyecto, países es una lista donde cada elemento es un diccionario que representa un país. Se recorre con for, se ordena con sorted() y se modifica con .append() y .clear().

```
países = cargar_paises()    # lista de diccionarios
países.append(nuevo_pais)  # agregar
países.clear()             # vaciar
```

Diccionario

Un diccionario almacena pares clave-valor. Cada país es un diccionario con cuatro claves: nombre, población, superficie y continente. Permite acceder a cada campo por nombre en lugar de por índice.

```
pais = {"nombre": "Argentina", "poblacion": 45376763,
        "superficie": 2780400, "continente": "America"}
print(pais["nombre"])    # Argentina
pais["poblacion"] = 46000000 # actualizar
```

Funciones

Una función es un bloque reutilizable que realiza una tarea específica (def). En el proyecto hay 13 funciones, cada una con una sola responsabilidad: crear_csv(), cargar_paises(), guardar_paises(), mostrar_lista(), agregar_pais(), actualizar_pais(), buscar_pais(), buscar_indice(), filtrar_paises(), ordenar_paises(), mostrar_estadisticas(), mostrar_menu() y main().

Condicionales

Las estructuras if / elif / else controlan el flujo según condiciones. Se usan para seleccionar opciones del menú, validar campos vacíos, verificar números positivos y detectar nombres duplicados.

```
if nombre == "":
    print(" [ERROR] El nombre no puede estar vacio.")
elif buscar_indice(paises, nombre) is not None:
    print(" [ERROR] Ya existe un pais con ese nombre.")
else:
    break
```

Ordenamientos

sorted() devuelve una nueva lista ordenada. Con key=lambda se elige el campo de comparación y con reverse=True el orden es descendente. Una vez ordenada, se actualiza la lista en memoria y se guarda en el CSV.

```
ordenados = sorted(paises, key=lambda p: p[campo], reverse=descendente)
paises.clear()
for p in ordenados:
    paises.append(p)
guardar_paises(paises)
```

Estadísticas Básicas

Se calculan con bucles for: el mayor y menor recorriendo la lista y comparando, el promedio acumulando la suma y dividiendo por len(), y el conteo por continente usando un diccionario contador.

```
mayor = paises[0]
for p in paises:
    if p["poblacion"] > mayor["poblacion"]: mayor = p
total = 0
for p in paises: total += p["poblacion"]
promedio = total // len(paises)
```

Archivos CSV

CSV es un formato de texto plano para datos tabulares. Python incluye el modulo csv con DictReader (leer) y DictWriter (escribir). La ruta del archivo se calcula con os.path para que siempre apunte a la misma carpeta que el .py.

```
CARPETA = os.path.dirname(os.path.abspath(__file__))
ARCHIVO_CSV = os.path.join(CARPETA, "paises.csv")
with open(ARCHIVO_CSV, newline="", encoding='utf-8') as f:
    lector = csv.DictReader(f)
    for fila in lector: paises.append(...)
```

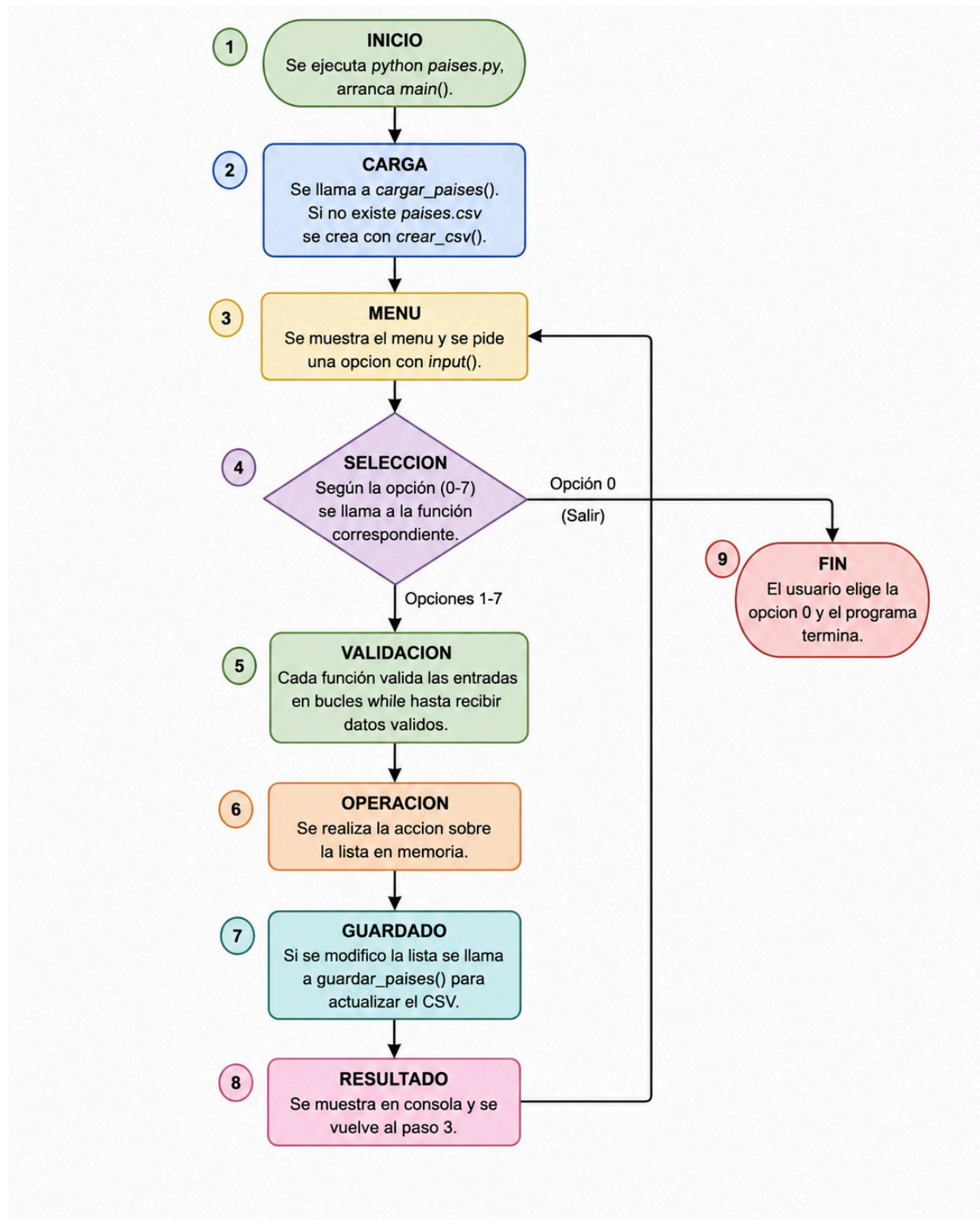
Decisiones Técnicas y Arquitectura

El sistema sigue el principio de responsabilidad única: cada función hace exactamente una cosa.

Funcion	Responsabilidad
crear_csv()	Crea paises.csv con los 8 paises iniciales si no existe
cargar_paises()	Lee el CSV y retorna la lista. Si no existe, llama a crear_csv()
guardar_paises()	Sobreescribe el CSV completo con el estado actual de la lista
agregar_pais()	Valida y agrega un pais nuevo, luego guarda
actualizar_pais()	Busca por nombre y actualiza poblacion y/o superficie
buscar_pais()	Busqueda parcial por nombre (case-insensitive)
buscar_indice()	Retorna el indice de un pais por nombre exacto o None
filtrar_paises()	Filtra por continente, rango poblacion o superficie
ordenar_paises()	Ordena, actualiza la lista en memoria y guarda en CSV
mostrar_estadisticas()	Calcula mayor/menor, promedios y conteo por continente
mostrar_lista()	Imprime la lista en formato de tabla en consola
main()	Carga el dataset y ejecuta el bucle del menu

- Persistencia inmediata: se guarda el CSV cada vez que hay un cambio, no solo al salir.
- Ruta dinamica: `os.path.abspath(__file__)` garantiza que el CSV se cree junto al .py.
- Creación automática: si paises.csv no existe, el programa lo genera con 8 países.

Desarrollo del Proyecto



Capturas de Ejecución

Ejemplo de la ejecución de la opción 1 (Mostrar Países):

```
=====
GESTION DE PAISES - Programacion 1 UTN
=====
1. Mostrar todos los paises
2. Agregar un pais
3. Actualizar poblacion / superficie
4. Buscar pais por nombre
5. Filtrar paises
6. Ordenar paises
7. Ver estadisticas
0. Salir
-----
Elegí una opcion: 1

-----
Nº    Nombre                Poblacion    Superficie km2  Continente
-----
1     Canada                 41,500,000    9,984,670      America
2     Argentina               45,376,763    2,780,400      America
3     Espana                   47,326,687    505,990        Europa
4     Francia                  67,391,582    551,695        Europa
5     Alemania                 83,149,300    357,022        Europa
6     Mexico                   130,262,216   1,964,375      America
7     Brasil                   213,993,437   8,515,767      America
8     Estados Unidos          331,893,745   9,833,517      America
-----
```

Ejemplo de la ejecución de la opción 4 (Busqueda):

```
=====
GESTION DE PAISES - Programacion 1 UTN
=====
1. Mostrar todos los paises
2. Agregar un pais
3. Actualizar poblacion / superficie
4. Buscar pais por nombre
5. Filtrar paises
6. Ordenar paises
7. Ver estadisticas
0. Salir
-----
Elegí una opcion: 4

-- BUSCAR PAIS --
Nombre o parte del nombre: Francia
Se encontraron 1 resultado(s):
```


TRABAJO PRÁCTICO INTEGRADOR - PROGRAMACIÓN 1

Nº	Nombre	Poblacion	Superficie km2	Continente
1	Francia	67,391,582	551,695	Europa

Ejemplo de la ejecución de la opción 7 (Estadísticas):

```
=====
GESTION DE PAISES - Programacion 1 UTN
=====
1. Mostrar todos los paises
2. Agregar un pais
3. Actualizar poblacion / superficie
4. Buscar pais por nombre
5. Filtrar paises
6. Ordenar paises
7. Ver estadisticas
0. Salir
-----
Elegí una opcion: 7
```

```
-- ESTADISTICAS --

Total de paises: 8

Mayor poblacion : Estados Unidos (331893745 hab.)
Menor poblacion : Canada (41500000 hab.)

Promedio de poblacion : 120111716 hab.
Promedio de superficie : 4311679 km2

Paises por continente:
  America: 5
  Europa: 3
```

Dificultades Encontradas:

- Ruta del CSV: el programa creaba el archivo en la carpeta de la terminal en lugar de la del .py. Se resolvió con `os.path.dirname(os.path.abspath(__file__))`.
- Persistencia del orden: `sorted()` no modifica la lista original. Se resolvió con `países.clear() + países.append()` + `guardar_países()`.
- Validación de entradas: fue necesario cubrir todos los casos posibles (vacío, no numérico, negativo, duplicado) con bucles `while`.
- Filas vacías en el CSV: se resolvió con `.strip()` en cada campo y `try/except` al convertir a entero.
- Búsqueda case-insensitive: se resolvió convirtiendo ambos strings a minúsculas con `.lower()` al comparar.

Conclusión:

Mediante la generación del TPI se permitió integrar todos los conceptos dados durante el cuatrimestre, en un proyecto real y funcional. La estructuras de lista de diccionarios resultó ideal para representar datos tabulares de forma clara y operable, las dificultades encontradas, especialmente en la gestión de rutas y la persistencia del orden, nos obligan a investigar funciones del módulo y el funcionamiento interno de `sorted()`, lo que representa un aprendizaje extra valioso.

Bibliografía / Webgrafía

Listas y Diccionarios (Estructura de datos):

[Python Software Foundation](#). Data Structures — Python Documentation.

Sección oficial sobre el funcionamiento, métodos y mutabilidad de estructuras de datos lineales y asociativas en Python.

Funciones:

[Python Software Foundation](#). Defining Functions — Python Documentation.

Guía oficial sobre la sintaxis de declaración (def), el uso de argumentos y el retorno de valores en bloques modulares de código.

Estructuras de Control Condicionales:

[Python Software Foundation](#). More Control Flow Tools: if Statements.

Documentación oficial sobre la lógica de ramificación del flujo de un programa mediante las estructuras if, elif y else.

Ordenamiento:

[Abel Garrido / Python in Plain English](#). Python Sorting Algorithms: Bubble Sort.

Análisis conceptual y técnico sobre la lógica de comparación lineal e intercambio de posiciones en el algoritmo de ordenamiento de burbuja.

Archivo CSV:

[Python Software Foundation](#) (Español). csv — CSV File Reading and Writing.

Documentación oficial en español del módulo nativo para lectura y escritura de archivos planos estructurados por filas y columnas.

Links del Proyecto

Link del repositorio github:

<https://github.com/Antolopez-06/TPI-Programaci-n1-Lopez-Antonella>

Link del Video Explicativo: <https://www.youtube.com/watch?v=uP8VvKaFW8c>

Link del Docs:

<https://docs.google.com/document/d/18p4vAOjFMN4RbFrHoZcREJztLed-MRtwuBtehjQQzCQ/edit?tab=t.0>